

ADC0832

8 位串行 A/D 转换器 ADC0832

1. 功能特点

ADC0832是NS(National Semiconductor)公司生产的串行接口8位A/D转换器，通过三线接口与单片机连接，功耗低，性能价格比较高，适宜在袖珍式的智能仪器仪表中使用。ADC0832为8位分辨率A/D转换芯片，其最高分辨可达256级，可以适应一般的模拟量转换要求。芯片具有双数据输出可作为数据校验，以减少数据误差，转换速度快且稳定性能强。独立的芯片使能输入，使多器件连接和处理器控制变得更加方便。通过DI 数据输入端，可以轻易的实现通道功能的选择。其主要特点如下：

- 8 位分辨率，逐次逼近型，基准电压为 5V；
- 5V 单电源供电；
- 输入模拟信号电压范围为 0~5V；
- 输入和输出电平与 TTL 和 CMOS 兼容；
- 在 250KHZ 时钟频率时，转换时间为 32us；
- 具有两个可供选择的模拟输入通道；
- 功耗低，15mW。

2. 外部引脚及其说明

ADC0832 有 DIP 和 SOIC 两种封装，DIP 封装的 ADC0832 引脚排列如图 6.21 所示。

各引脚说明如下：

- CS——片选端，低电平有效。
- CH0, CH1——两路模拟信号输入端。
- DI——两路模拟输入选择输入端。
- DO——模数转换结果串行输出端。
- CLK——串行时钟输入端。
- Vcc/REF——正电源端和基准电压输入端。
- GND——电源地。

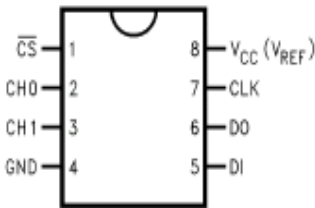


图 6.21 ADC0832 引脚图

3. 单片机对ADC0832 的控制原理

一般情况下ADC0832与单片机的接口应为4条数据线，分别是CS、CLK、DO、DI。但由于DO端与DI端在通信时并未同时有效并与单片机的接口是双向的，所以电路设计时可以将DO和DI 并联在一根数据线上使用。当ADC0832未工作时其CS输入端应为高电平，此时芯片禁用，CLK 和DO/DI 的电平可任意。当要进行A/D转换时，须先将CS端置于低电平并且保持低电平直到转换完全结束。此时芯片开始转换工作，同时由处理器向芯片时钟输入端CLK提供时钟脉冲，DO/DI端则使用DI端输入通道功能选择的数据信号。在第1个时钟脉冲到来之前DI端必须是高电平，表示启动位。在第2、3个时钟脉冲到来之前DI端应输入2位数据用于选择通道功能，其功能项见表6.4。

输入形式	配置位		选择通道	
	CH0	CH1	CH0	CH1
差分输入	0	0	+	-
	0	1	-	+
单端输入	1	0	+	
	1	1		+

表 6.4 ADC0832 配置位

如表6.4所示，当配置位2位数据为1、0时，只对CH0 进行单通道转换。当配置2位数据为1、1时，只对CH1进行单通道转换。当配置2位数据为0、0时，将CH0作为正输入端IN+，CH1 作为负输入端IN-进行输入。当配置2位数据为0、1时，将CH0作为负输入端IN-，CH1 作为正输入端IN+进行输入。

到第 3 个时钟脉冲到来之后 DI 端的输入电平就失去输入作用，此后 DO/DI 端则开始利用数据输出 DO 进行转换数据的读取。从第 4 个时钟脉冲开始由 DO 端输出转换数据最高位 D7，随后每一个脉冲 DO 端输出下一位数据。直到第 11 个脉冲时发出最低位数据 D0，一个字节的数据输出完成。也正是从此位开始输出下一个相反字节的数据，即从第 11 个时钟脉冲输出 D0。随后输出 8 位数据，到第 19 个脉冲时数据输出完成，也标志着一次 A/D 转换的结束。最后将 CS 置高电平禁用芯片，直接将转换后的数据进行处理就可以了。图 6.22 为 ADC0832 时序图。

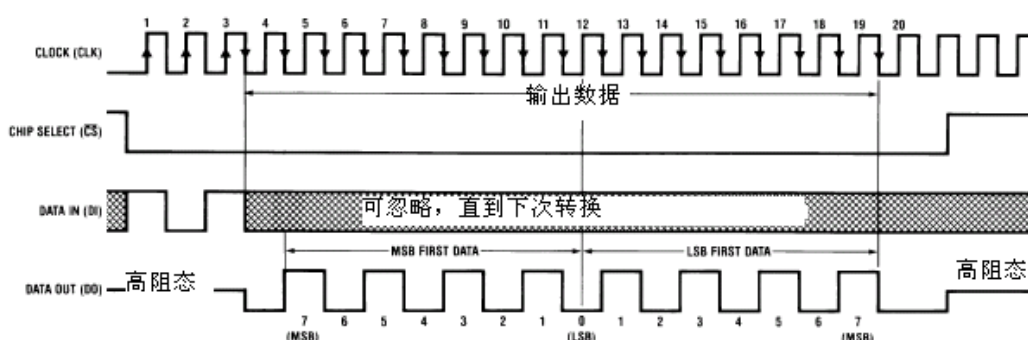


图 6.22 ADC0832 时序图

4. ADC0832 典型应用

(1) 单片机串行口方式 0 与 ADC0832 接口

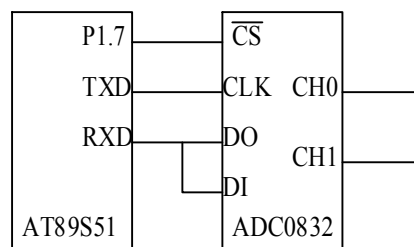


图 6.23 ADC0832 与单片机

如图 6.23 所示，AT89C51 的 P1.7 为片选信号端，TXD 是时钟信号输出端，RXD 为启动信号，模拟通道选择信号发送端以及 A/D 转换后输出数据的接收端。

ADC0832 的时钟频率最高为 400KHZ，单片机 AT89C51 晶振选用 4MHZ，在 TXD 端的输出频率为 $4\text{MHZ}/12=333\text{KHZ}$ ，符合要求。

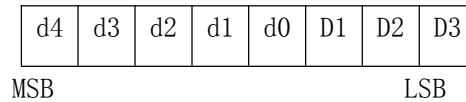
ADC0832 输出的串行数据共 15 位，由两段 8 位数据组成，前一段是最高位在先，后一段是最高位在后，两段数据的最低位共用。只有在时钟的下降沿，ADC0832 的串行数据才移出一位。由单片机控制时钟信号进行发送，并由 TXD 发出，以达到控制 ADC0832 输出数据位的目的。为了得到一系列完整的 8 位数据，单片机分两次采集含有不同位的数据，再合成一系列完整的数据。

当 REN=0 时，AT89C51 连续一次向 ADC0832 发送 8 个时钟脉冲，前 3 个脉冲发送的是启

动位和模拟通道选择位，共计 3 位；从第 4 个脉冲下降沿开始，ADC0832 发出转换数据 D7～D4（在脉冲上升沿单片机方可接收）。但由于 REN=0，单片机不予接收，丢失 D7～D4 数据。

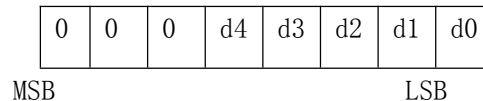
当 REN=1 时，单片机又向 ADC0832 连续发出 8 个时钟脉冲，其输出转换数据 D3，D2，D1，和 d0，d1，d2，d3，d4，存入累加器 A 形成如下结构：

累加器 A



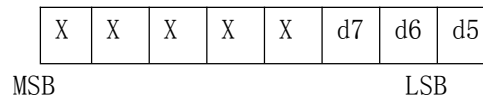
上述数据右移 3 位，并屏蔽掉高 3 位，暂存于寄存器 B，得到如下结构：

寄存器 B



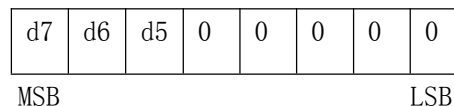
单片机第二次接收，可得到下列数据：

累加器 A



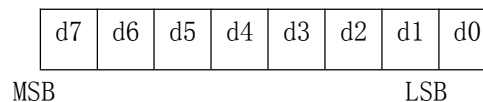
以上数据左移 5 位，并屏蔽低 5 位，送入累加器 A，得到如下结构：

累加器 A



进行 (A) + (B) → (A) 运算，得到如下结构：

累加器 A



从而得到一个完整的 8 位 A/D 转换结果。

根据图 6.23 对 CH1 通道的模拟输入信号实行 A/D 转换的程序如下：

```

CADA:   CLR    P1.7                ; CS=0
        MOV    SCON, #00H          ; 串行口方式 0，REN=0
        MOV    A, #07H             ; 通道配置位为 11，启动位为 1
        MOV    SBUF, A              ; 启动发送
LOOPA1: JNB    T1, LOOPA1           ; 发送等待
        MOV    SCON, #10H          ; REN=1，RI=0，启动接收
LOOPA2: JNB    R1, LOOPA2           ; 接受等待
        MOV    A, SBUF
        RR     A
        RR     A
        RR     A
        ANL    A, #1FH              ; 屏蔽高 3 位
        MOV    B, A
        MOV    SCON, #10H          ; 第二次启动接收
LOOPA3: JNB    RI, LOOPA3
        MOV    A, SBUF
    
```

```

RL      A
SWAP    A
ANL     A, #0E0H
ADD     A, B
SETB    P1.7
RET

```

（2）SPI 串行接口方式

SPI 是 MOTOROLA 公司推出的一种同步串行外设接口，允许 MCU 也各个厂家生产工具的标准外围设备直接接口，以串行方式交换信息。SPI 使用 4 条线与主机（MCU）连接：串行时钟 SCK，主机输入/从机输出数据线 SO，主机输出/从机输入数据线 SI 和低电平有效的从机选择线 CS。

SPI 串行扩展系统的主器件单片机，可以带有 SPI 接口，也可以不带 SPI 接口，但从器件必须具有 SPI 接口。

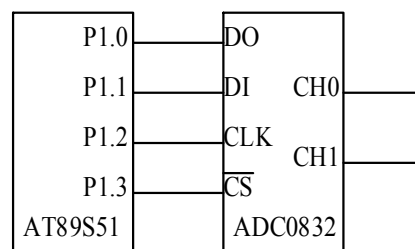


图 6.24 AT89S51 与 ADC0832 的 SPI 方式连接

ADC0832 具有 SPI 接口，图 6.24 为 AT89S51 与 ADC0832 的 SPI 串行接口方式，将 DO 和 DI 分别连接于 P1.0 和 P1.1。对 CH0 通道的模拟信号进行 A/D 转换，转换结果存于累加器 A 中。程序如下：

```

CADB:   CLR      P1.3                ; CS=0
        MOV      A, #03H            ; 起始位和配置位为 011
        MOV      R7, #03H
LOOPB1: CLR      P1.2                ; CLK=0
        RRC      A
        MOV      P1.1, C
        NOP
        SETB     P1.2                ; CLK=1
        DJNZ     R7, LOOPB1
        CLR      P1.2                ; 通道稳定脉冲
        NOP
        SETB     P1.2                ; CLK=1
        MOV      R7, #08H
LOOPB2: CLR      P1.2                ; CLK=0
        MOV      C, P1.0             ; 读入一位数据
        RLC      A
        SETB     P1.2                ; CLK=1
        DJNZ     R7, LOOPB2
        SETB     P1.3                ; CS=1

```

RET